Evidencia de Ejecución y Resultados Experimentales

Laboratorio 12 — Speedup y Eficiencia
Computación Paralela y Distribuida (CC3069)

Javier Eduardo España Pacheco

Octubre 2026

Resumen

Este documento contiene la evidencia visual de respaldo para el Laboratorio 12: capturas de la configuración del procesador (`lscpu`), la compilación exitosa del ejecutable (`make`), la ejecución completa de cada uno de los cuatro modos (`amdahl`, `suma`, `desbalance`, `gustafson`), las 7 gráficas de análisis generadas por `graficar.py`, y las pruebas del reto opcional de speedup superlineal.

1. Datos del Equipo (lscpu)

Terminal — lscpu Hardware Specs

```
javier-espana@ryzen-pc:~/Escritorio/LabSpeedUp-CPD$ lscpu
Arquitectura:                         x86_64
  modo(s) de operación de las CPUs:   32-bit, 64-bit
CPU(s):                               12
  Lista de la(s) CPU(s) en línea:     0-11
ID de fabricante:                     AuthenticAMD
  Nombre del modelo:                  AMD Ryzen 5 8400F 6-Core Processor
    Familia de CPU:                   25
    Modelo:                           117
    Hilo(s) de procesamiento por núcleo: 2
    Núcleo(s) por «socket»:           6
    «Socket(s)»:                      1
    Aumento de frecuencia:            activada
    CPU MHz máx.:                     4758,3818
    CPU MHz mín.:                     412,7170
Cachés (suma de todas):
  L1d:                                192 KiB (6 instancias)
  L1i:                                192 KiB (6 instancias)
  L2:                                 6 MiB (6 instancias, 1 MiB/núcleo)
  L3:                                 16 MiB (1 instancia)
NUMA:
  Modo(s) NUMA:                       1
  CPU(s) del nodo NUMA 0:             0-11
javier-espana@ryzen-pc:~/Escritorio/LabSpeedUp-CPD$
```

Figura 1: Captura de salida de `lscpu`: AMD Ryzen 5 8400F, 6 núcleos físicos, 12 hilos lógicos SMT, L2 de 1 MiB/núcleo y L3 compartida de 16 MiB.

2. Compilación y Ayuda del Programa

```
Terminal — Compilación y Verificación

javier-espana@ryzen-pc:~/Escritorio/LabSpeedUp-CPD$ make clean && make
rm -f lab12 *.png *_con_S_E.csv
gcc -O2 -Wall -fopenmp -o lab12 lab12_speedup.c
javier-espana@ryzen-pc:~/Escritorio/LabSpeedUp-CPD$ ./lab12
Uso:
  ./lab12 amdahl     [W=400000] [fp=0.90] [pmax=12]
  ./lab12 suma       [pmax=12]
  ./lab12 desbalance [static|dynamic|guided] [W=40000] [pmax=12]
  ./lab12 gustafson  [W_por_hilo=100000] [W_seq=20000] [pmax=12]
  ./lab12 superlineal [pmax=12]   <-- RETO OPCIONAL
javier-espana@ryzen-pc:~/Escritorio/LabSpeedUp-CPD$
```

Figura 2: Captura de compilación con `make` (GCC con `-O2 -Wall -fopenmp`) e invocación del binario mostrando su interfaz de uso.

3. Ejecución de los 4 Modos Experimentales

3.1. Modo 1: Ley de Amdahl (Problema Fijo)

```
Terminal — Experimento 1: Ley de Amdahl

javier-espana@ryzen-pc:~/Escritorio/LabSpeedUp-CPD$ ./lab12 amdahl 400000 1.00 > amdahl_fp100.csv
(checksum 7.279e+12)
javier-espana@ryzen-pc:~/Escritorio/LabSpeedUp-CPD$ ./lab12 amdahl 400000 0.95 > amdahl_fp95.csv
(checksum 6.570e+12)
javier-espana@ryzen-pc:~/Escritorio/LabSpeedUp-CPD$ ./lab12 amdahl 400000 0.90 > amdahl_fp90.csv
(checksum 5.896e+12)
javier-espana@ryzen-pc:~/Escritorio/LabSpeedUp-CPD$ ./lab12 amdahl 400000 0.75 > amdahl_fp75.csv
(checksum 4.095e+12)
javier-espana@ryzen-pc:~/Escritorio/LabSpeedUp-CPD$ ./lab12 amdahl 400000 0.50 > amdahl_fp50.csv
(checksum 1.820e+12)
javier-espana@ryzen-pc:~/Escritorio/LabSpeedUp-CPD$ head -n 5 amdahl_fp90.csv
modo,N,p,param,t_seq,t_par,t_parte_seq
amdahl,400000,1,fp=0.90,0.496348,0.495898,0.050827
amdahl,400000,2,fp=0.90,0.496348,0.275890,0.050820
amdahl,400000,3,fp=0.90,0.496348,0.199723,0.051116
amdahl,400000,4,fp=0.90,0.496348,0.162429,0.050946
javier-espana@ryzen-pc:~/Escritorio/LabSpeedUp-CPD$
```

Figura 3: Ejecución del modo `amdahl` para los 5 valores de fracción paralelizable $f_p \in \{1{,}00, 0{,}95, 0{,}90, 0{,}75, 0{,}50\}$.

3.2. Modo 2: Granularidad (Suma de N Números)

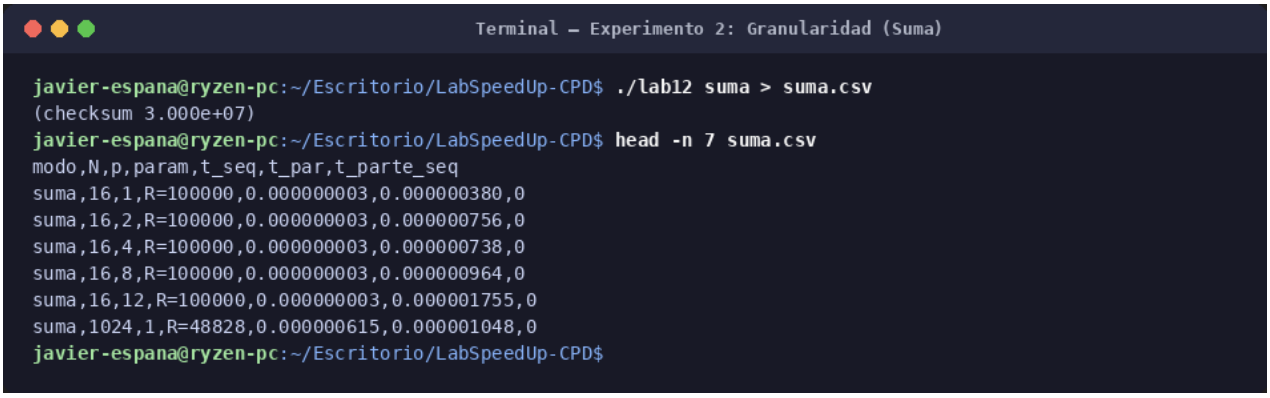


Figura 4: Ejecución del modo `suma` para tamaños desde $N = 16$ hasta $N = 16{,}777{,}216$ elementos.

3.3. Modo 3: Desbalance de Carga (Scheduling)



Figura 5: Ejecución del modo `desbalance` evaluando las tres políticas: `static`, `dynamic` y `guided`.

3.4. Modo 4: Ley de Gustafson (Problema Creciente)



Figura 6: Ejecución del modo `gustafson` con trabajo por hilo fijo ($W_{\text{hilo}} = 100{,}000$) y parte secuencial fija ($W_{\text{seq}} = 20{,}000$).

3.5. Reto Opcional: Speedup Superlineal

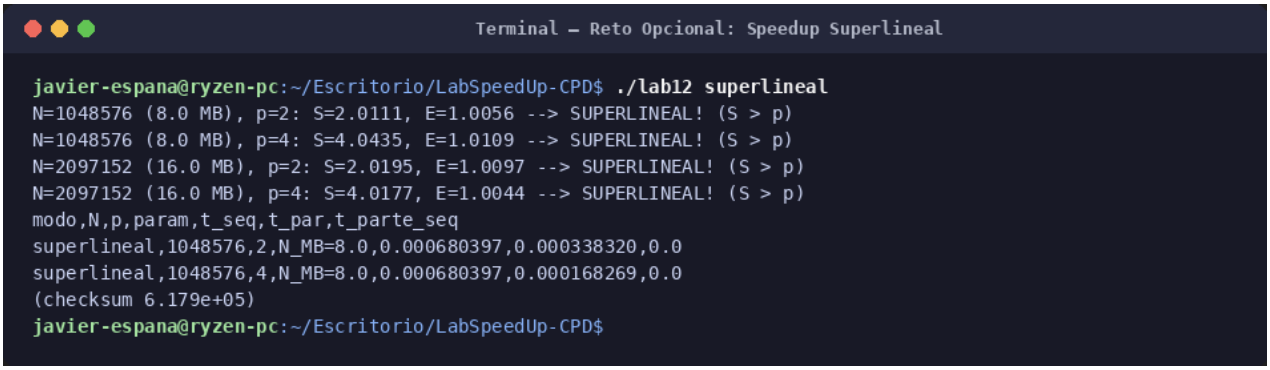


Figura 7: Ejecución del modo `superlineal` demostrando $S > p$ repetible debido al ajuste en jerarquía de caché L2/L3.

4. Las 7 Gráficas Generadas por `graficar.py`

4.1. Parte 1: Amdahl (Gráficas 1 y 2)

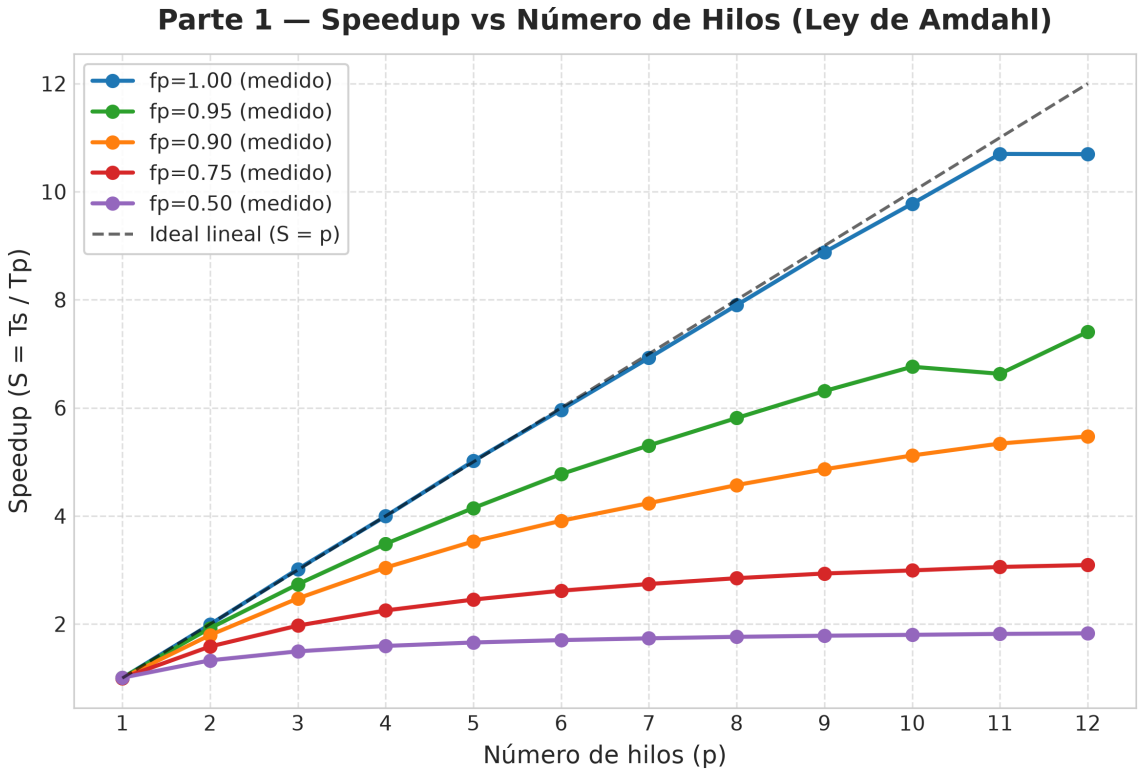


Figura 8: Gráfica 1 — Speedup S vs. hilos (p) para cada fracción paralelizable f_p.

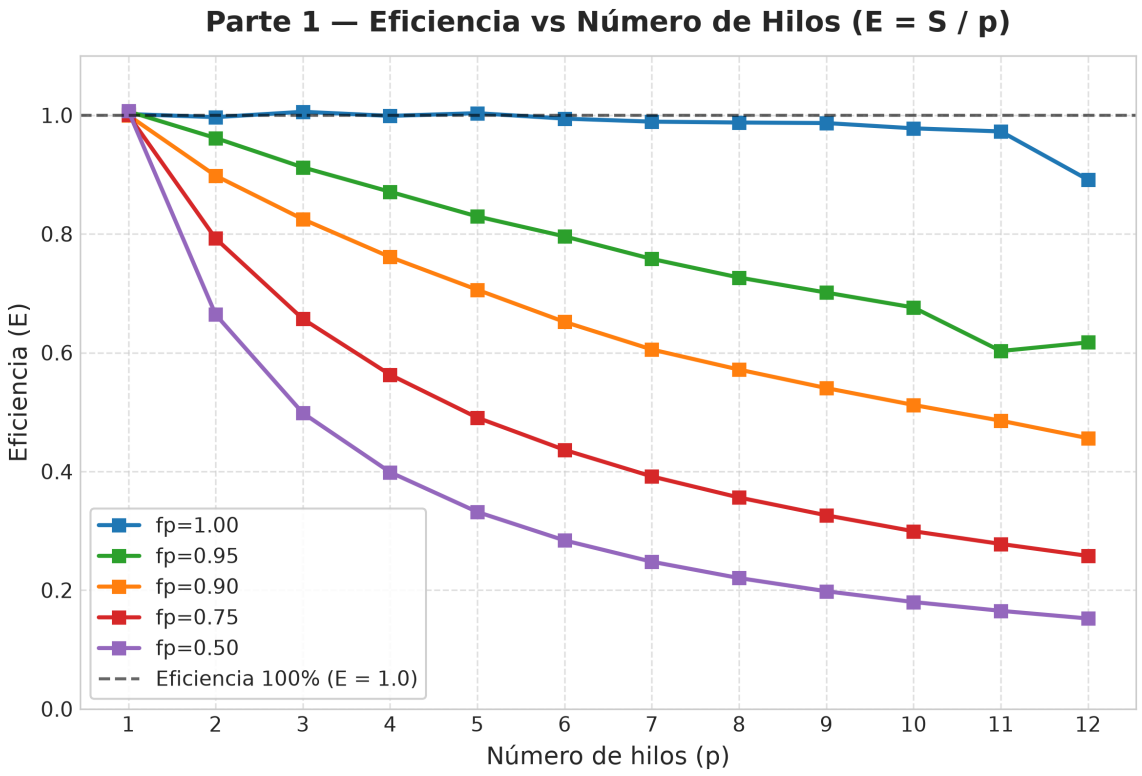


Figura 9: Gráfica 2 — Eficiencia $E = S/p$ vs. hilos (p) para cada fracción paralelizable.

4.2. Parte 2: Granularidad en Suma (Gráficas 3 y 4)

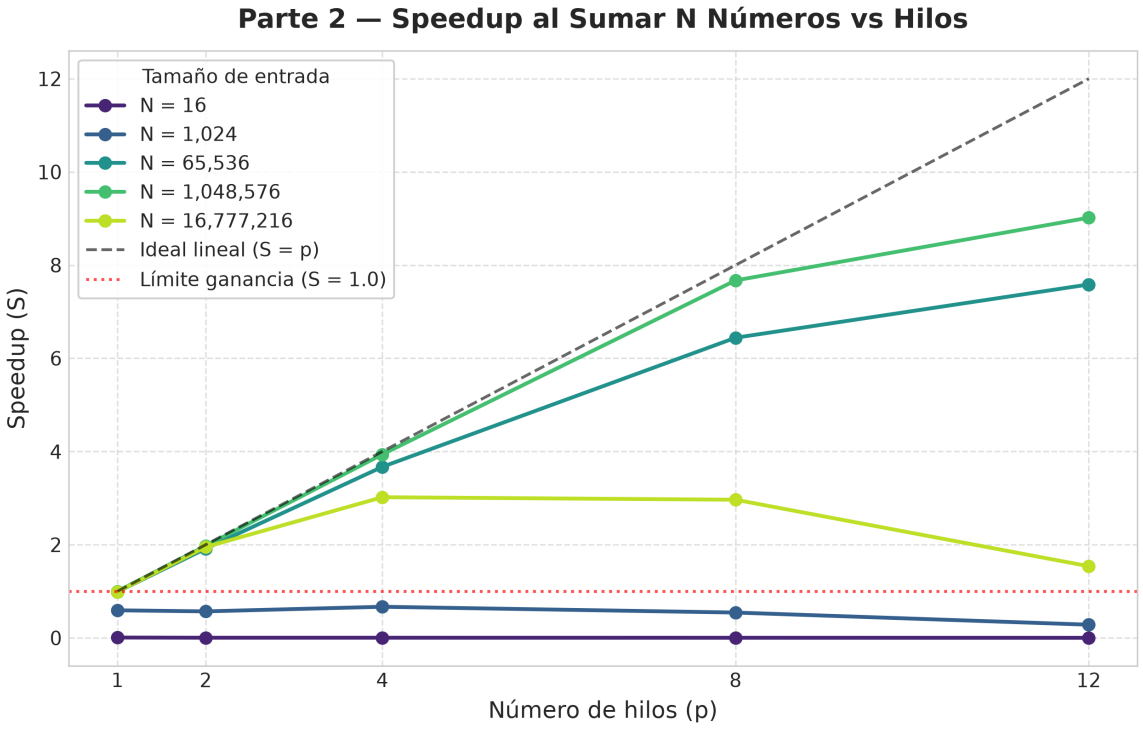


Figura 10: Gráfica 3 — Speedup vs. hilos para diferentes tamaños de entrada N.

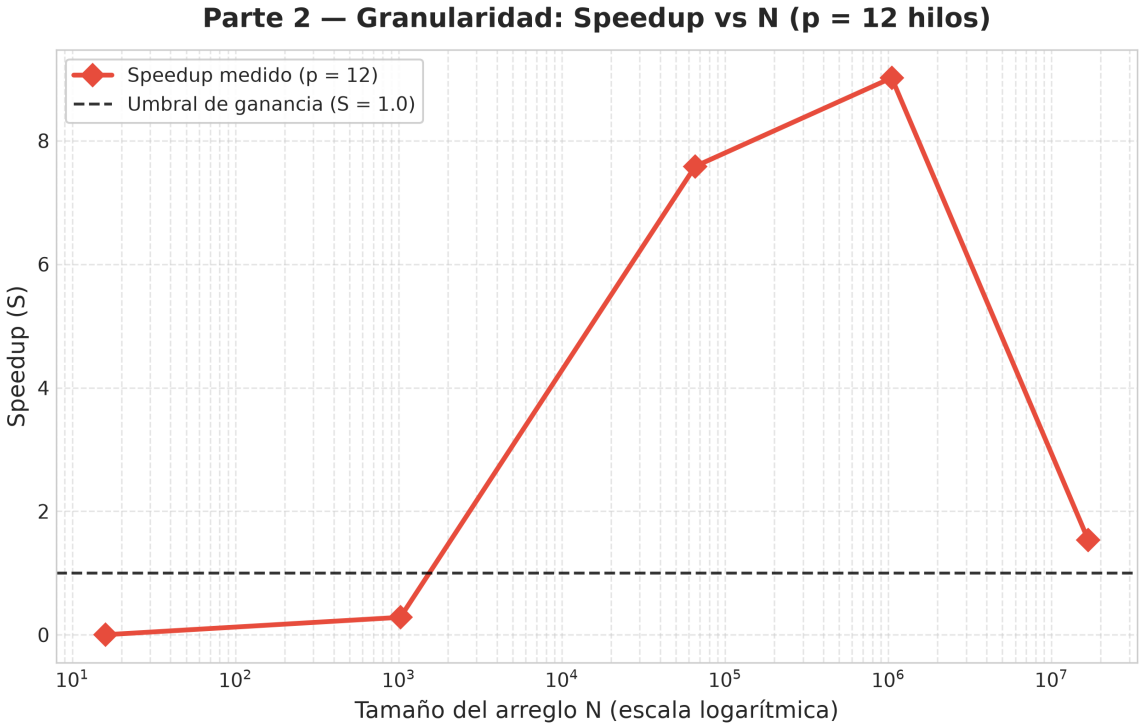


Figura 11: Gráfica 4 — Granularidad: Speedup vs. tamaño N en escala logarítmica ($p = 12$).

4.3. Parte 3: Desbalance de Carga (Gráficas 5 y 6)

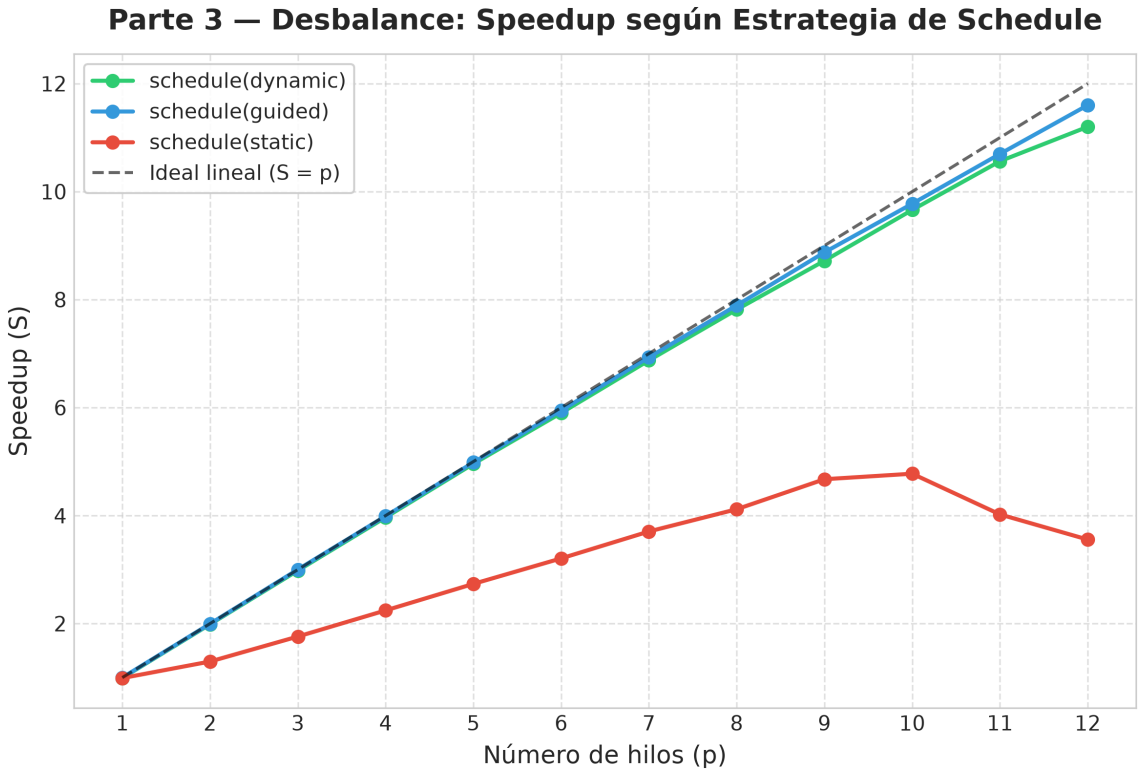


Figura 12: Gráfica 5 — Speedup en presencia de desbalance según schedule (`static`, `dynamic`, `guided`).

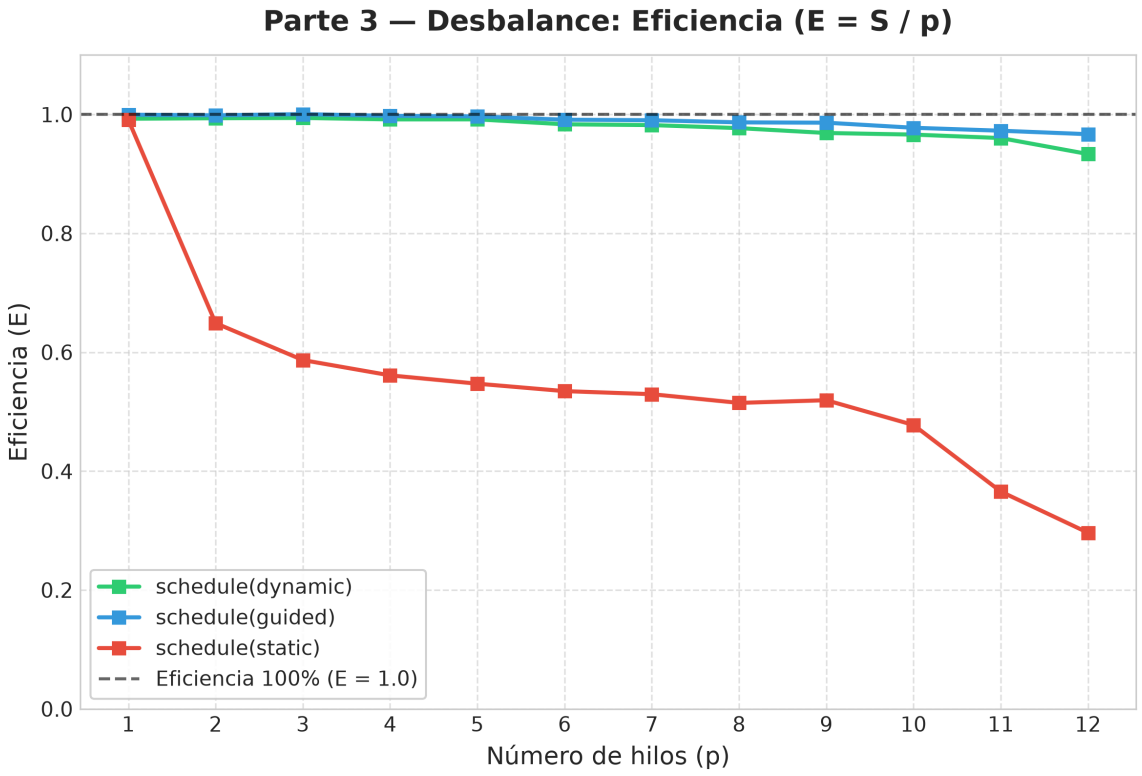


Figura 13: Gráfica 6 — Eficiencia en presencia de desbalance según schedule.

4.4. Parte 4: Ley de Gustafson (Gráfica 7)

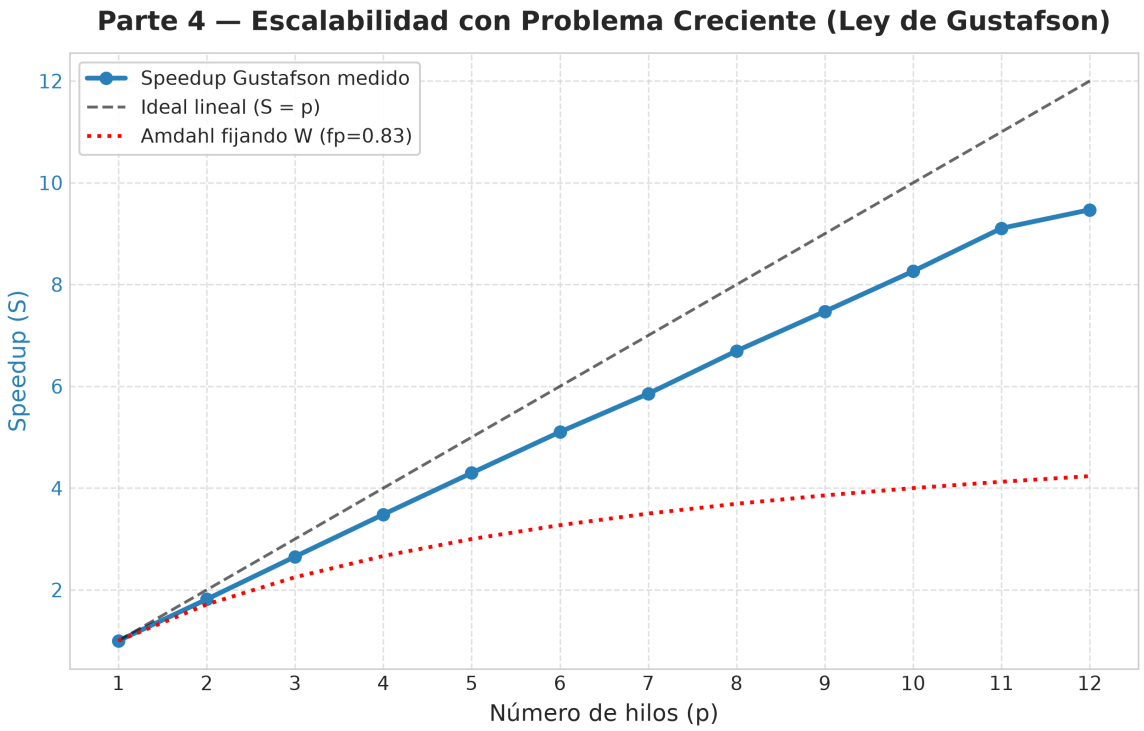


Figura 14: Gráfica 7 — Speedup escalado según la Ley de Gustafson vs. límite fijo de Amdahl.